

Docker Best Practices:

1. Use Official Docker Image as Base Image

- Cleaner Dockerfile
- Official Docker Image
- Verified and already built with best practices

The diagram illustrates the correct and incorrect ways to set a base image in a Dockerfile. At the top, a blue banner reads "Use Official Docker Images as Base Image". Below this, a Dockerfile icon is shown with two bullet points: "cleaner Dockerfile" and "official Docker Image".

On the left, a terminal window shows an incorrect Dockerfile snippet:

```
FROM ubuntu
```

This is marked with a large red 'X'. A callout box points to it, stating "Base operating system image". Below the terminal, another red 'X' is shown with a callout box stating "Install packages by yourself". The terminal content includes:

```
RUN apt-get update && apt-get install -y \
    node \
    && rm -rf /var/lib/apt/lists/*
```

On the right, a terminal window shows the correct Dockerfile snippet:

```
FROM node
```

This is marked with a green checkmark. A callout box points to it, stating "Use official 'node' image as base image".

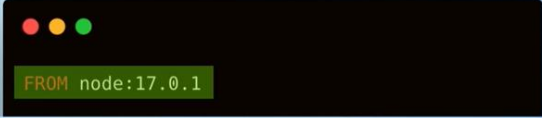
2. Use specific Image tag

- a. It will use the latest tag
- b. FROM node => From node: latest
- c. You might get different docker image versions
- d. Might break stuff
- e. Latest tag is unpredictable

 **Use Specific Image Versions**




DO NOT use a random latest tag




Fixate the version



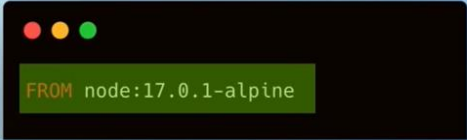
The more specific, the better

3. Use Small-Sized Official Images

- a. Smaller Images
- b. Less storage spaces
- c. Transfer Images faster
- d. Minimize surface attacks
 - i. Larger Image Size
 - ii. More security Vulnerabilities
 - iii. Introduce unnecessary security issues from the beginning




Could be based on full-blown OS




Use Image based on a leaner and smaller OS distro

4. Optimizing Caching Image Layers

- a. docker image history <image id>

Optimize Caching Image Layers

Dockerfile Instructions

- 1) Use Node Alpine Image as Base Image
- 2) Go to /app folder
- 3) Copy project files into the Docker Image
- 4) Install project dependencies with "npm install"



```
FROM node:17.0.1-alpine

WORKDIR /app

COPY myapp /app

RUN npm install --production

CMD ["node", "src/index.js"]
```

Why isn't it used from cache?



- Once a layer changes, all following layers are re-created as well



Local Cache

Cache invalidated
Cache invalidated
Cache invalidated

```
FROM node:17.0.1-alpine ✓ From Cache

WORKDIR /app ✓ From Cache

COPY myapp /app ✗

RUN npm install --production ✗

CMD ["node", "src/index.js"] ✗
```

Optimize Caching Image Layers

Optimize caching for "npm install" layer

- **DO NOT RE-RUN:** when project files changes
- **RE-RUN:** when package.json file changes

copy project files
install dependencies
copy package.json
set working directory
pull node alpine image



Restructure Dockerfile instructions

```
FROM node:17.0.1-alpine

WORKDIR /app

COPY package.json package-lock.json .

RUN npm install --production

COPY myapp /app

CMD ["node", "src/index.js"]
```

Optimize Caching Image Layers

Optimize caching for "npm install" layer

- ▶ **DO NOT RE-RUN:** when project files changes
- ▶ **RE-RUN:** when package.json file changes

Update
server.js file

copy project files	Cache invalidated
install dependencies	from Cache
copy package.json	from Cache
set working directory	from Cache
pull node alpine image	from Cache

```

FROM node:17.0.1-alpine

WORKDIR /app

COPY package.json package-lock.json .

RUN npm install --production

COPY myapp /app

CMD ["node", "src/index.js"]

```

✓

**Order Dockerfile commands
from least to most frequently changing**

Least changed

↓

Most Frequently changed

✓

Take advantage of caching

```

FROM node:17.0.1-alpine

WORKDIR /app

COPY package.json package-lock.json .

RUN npm install --production

COPY myapp /app

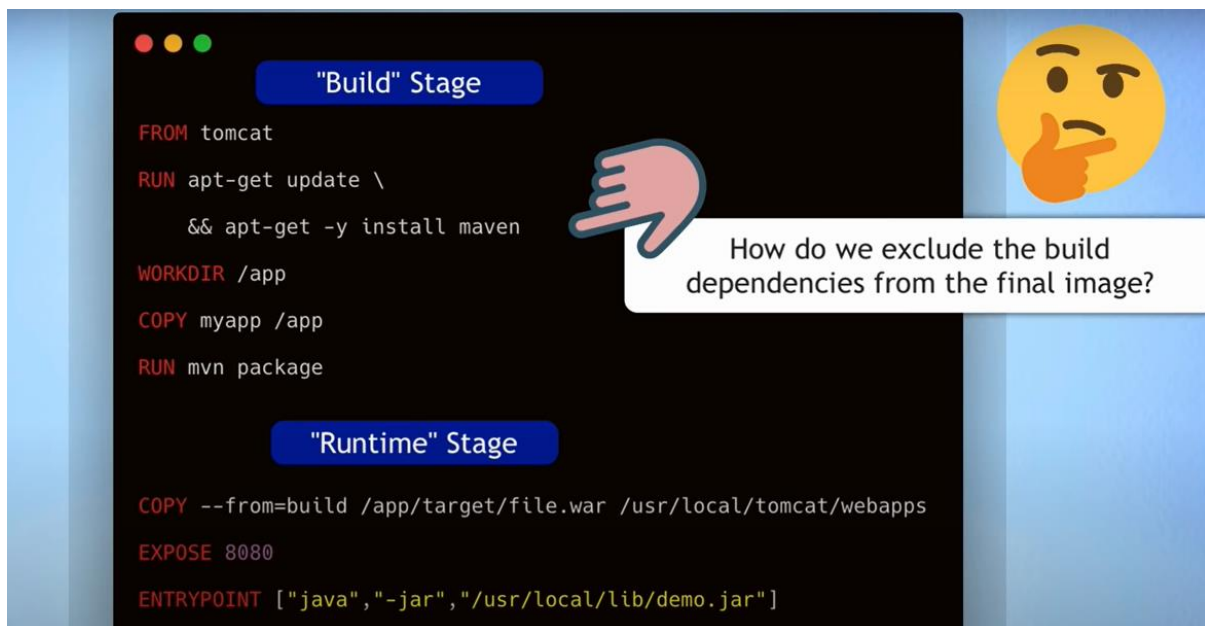
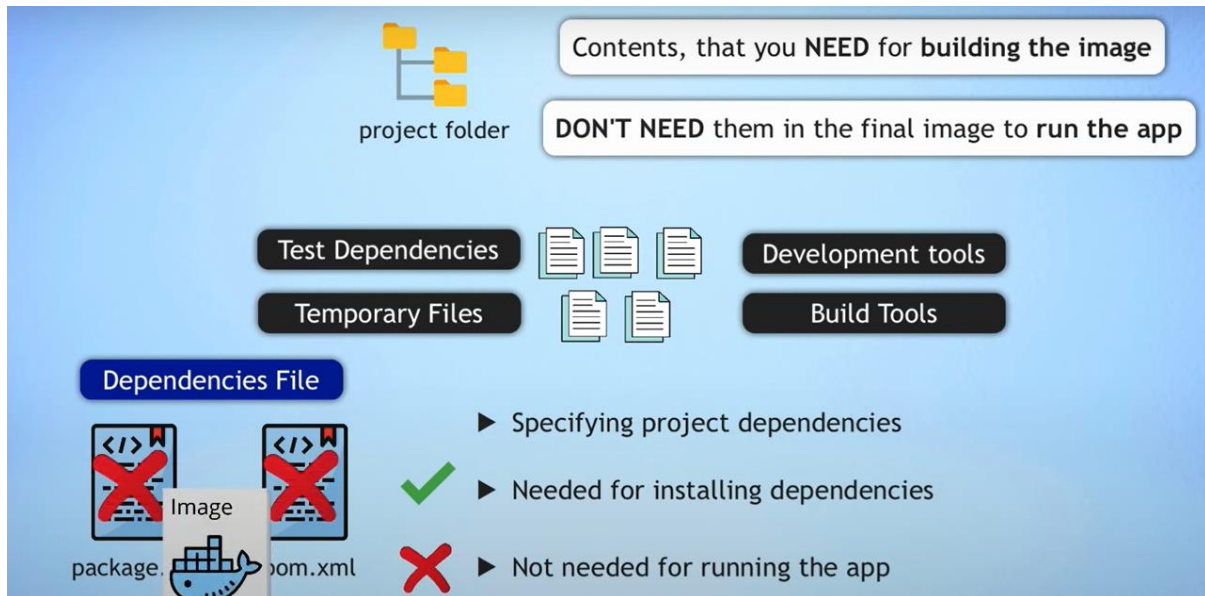
CMD ["node", "src/index.js"]

```

5. Use `.dockerignore` to explicitly exclude files and folders
 - a. We don't need everything inside the Docker image
 - b. Auto-generated folders (like target, build)
 - c. README.md
 - d. In Order to
 - i. Reduce Image Size
 - ii. Prevent Unintended secrets exposure
 - iii. Create `.dockerignore` file in the root directory
 - iv. Ignore `.git` and `.cache`
6. Make use of "multi-Stage" build
 - a. How do we exclude the build dependency from the final image?

b. Build Stage/ Runtime Stage

c. Contents, that you need for building the image. DON'T NEED them in the final image to run the app



- You can name your stages with "AS <name>"

- 1st stage: Builds Java App

1st

Dockerfile with 2 Build Stages

```
# Build stage
FROM maven AS build
WORKDIR /app
COPY myapp /app
RUN mvn package

#Run stage
FROM tomcat
COPY --from=build /app/target/file.war /usr/local/tomcat/webapps/
```

- Each **FROM** instruction starts a new build stage
- You can selectively copy artifacts from one stage to another

2nd

Dockerfile with 2 Build Stages

```
# Build stage
FROM maven AS build
WORKDIR /app
COPY myapp /app
RUN mvn package

#Run stage
FROM tomcat
COPY --from=build /app/target/file.war /usr/local/tomcat/webapps/
```



Leaving everything behind you don't want in the final image



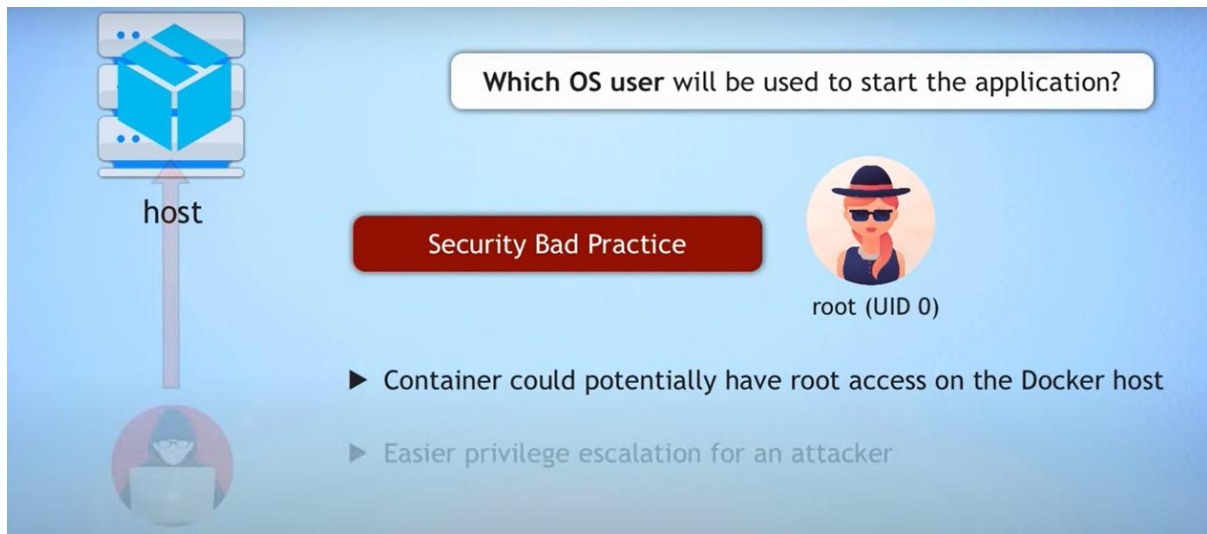
final image

Dockerfile with 2 Build Stages

```
# Build stage
FROM maven AS build
WORKDIR /app
COPY myapp /app
RUN mvn package

#Run stage
FROM tomcat
COPY --from=build /app/target/file.war /usr/local/tomcat/webapps/
...
```

7. Use the Least Privileged User



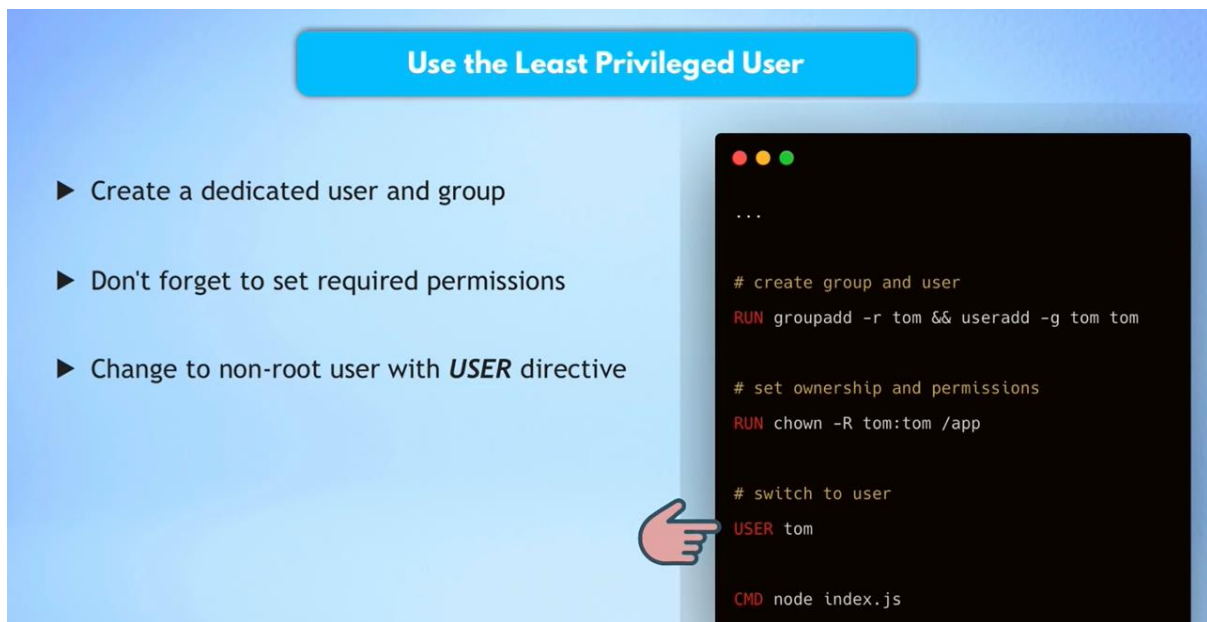
Which OS user will be used to start the application?

host

Security Bad Practice

root (UID 0)

- ▶ Container could potentially have root access on the Docker host
- ▶ Easier privilege escalation for an attacker



Use the Least Privileged User

- ▶ Create a dedicated user and group
- ▶ Don't forget to set required permissions
- ▶ Change to non-root user with **USER** directive

```
...  
  
# create group and user  
RUN groupadd -r tom && useradd -g tom tom  
  
# set ownership and permissions  
RUN chown -R tom:tom /app  
  
# switch to user  
USER tom  
  
CMD node index.js
```

8. Scan Your Image for Vulnerabilities using docker scan myapp:1.0

- How to validate the built Image for security vulnerabilities?
- Docker uses Snyk service for the vulnerability scan.
- You can configure DockerHub, to scan the Image automatically when the image gets pushed to the Docker repo
- Integrate in CI/CD